

Procesadores gráficos y Aplicaciones en Tiempo Real I

Box filter de una imagen

Descripción

En esta práctica se abordará el problema de realizar un box filter específico a una imagen de forma paralela (adaptada de la práctica de blur de *Udacity* del curso *Introduction to Parallel Programming*). En este caso, el filtro de la imagen servirá tanto para mejorar la nitidez de una imagen, como suavizarla, o encontrar sus bordes. La idea final es permitir una mejor identificación de los objetos que se encuentren en la imagen.

Para ello en principio se va a realizar un filtrado en el dominio del espacio llevándose a cabo directamente sobre los píxeles de la imagen. En este proceso se relaciona, para todos y cada uno de los puntos de la imagen, un conjunto de píxeles próximos al píxel objetivo sobre los que se aplica un filtro. Para ello se realiza una convolución del filtro (f) sobre la imagen (w), donde cada píxel de la nueva imagen (g) se obtiene mediante el sumatorio de la multiplicación del filtro por los píxeles contiguos:

$$g(x,y) = \sum \sum f(i,j) w(i,j)$$

En concreto, imaginemos que tenemos una matriz cuadrada de pesos que representa el filtro. Para cada píxel de la imagen w , superponemos esta matriz cuadrada de pesos de manera que el centro de la matriz se alinee con el píxel a analizar. Para calcular el valor del nuevo píxel, multiplicamos cada par de números que se alinean. En otras palabras, multiplicamos cada peso con el valor del píxel que se ha superpuesto. Por último, sumamos todos los números multiplicados y asignamos ese valor como nuevo píxel. Este proceso se debe realizar para todos los píxeles de la imagen. Esta forma de filtrado es llevada a cabo por herramientas comerciales, por ejemplo, Photoshop bajo el nombre "Custom Filter". La idea de la práctica es su realización de forma paralela, y luego utilizar lo aprendido para realizar un filtro complejo.

Para la implementación del box filter se cuenta con un código de apoyo basado en la librería OpenCV. OpenCV es una librería BSD multiplataforma de visión artificial para el tratamiento de imágenes, y se utiliza para abrir, salvar, ... y gestionar diferentes tipos de imágenes. Los alumnos solo deben modificar el fichero *func.cu* para incluir su solución, aunque deberá preparar el proyecto de la forma adecuada para la utilización de OpenCV 4. Para ello, en caso necesario de utilización en Windows, se puede modificar en Visual Studio:

- Linker -> Input -> Additional Dependencies (en este caso añadir *opencv_world4XXd.lib* *d* significa librería para debug y *XX* versión de librería. Si se utiliza en modo Release se debe utilizar la librería normal *opencv_world4XX.lib* sin *d*)
- Configuration Properties -> VC++ Directories -> Include Directories. Se debe incluir *OPENCV_DIR/build/include* donde *OPENCV_DIR* es el directorio principal de OpenCV
- Configuration Properties -> VC++ Directories -> Library Directories. Se debe incluir *OPENCV_DIR/build/x64/vcVV/lib* donde *vcVV* corresponde a la última versión posible.

En concreto en el código de apoyo, cómo las imágenes en color están formadas por varios canales, se separan los diferentes canales de color de manera que cada color se almacene de forma separada en lugar de estar intercalados. Esto simplifica el trabajo a realizar, ya que el kernel en CUDA trabajará sólo sobre un único canal de color, aunque habrá que ejecutarlo varias veces para tratar cada canal por separado (el código que recombina los resultados para cada color se encuentra como código de apoyo). El objetivo es rellenar el kernel llamado *box_filter* para realizar el filtrado con la matriz de pesos facilitada como *filter* (de *filterWidth* dimensiones cuadradas) sobre el canal de color especificado por *InputChannel*, escribiendo el resultado en *OutputChannel*. He aquí un ejemplo del cálculo que se debe realizar para un único píxel (la matriz de pesos estará alineada para el centro de la caja):

Imagen:

Filtro:

0	-0.25	0
-0.25	2	-0.25
0	-0.25	0

1	2	5	2	0	3
3	2	5	1	6	0
4	3	6	2	1	4
0	4	0	3	4	5
4	5	6	2	3	1

Valor obtenido = $0*2 - 0.25*5 + 0*1 - 0.25*3 + 2*6 - 0.25*2 + 0*4 - 0.25*0 + 0*3 = 9.5$

Es necesario tener en cuenta que el resultado obtenido puede ser menor que 0 o mayor que 255 por lo que es necesario tratar dichos casos para que la imagen final sea correcta, en caso de estar utilizando RGB.

Para su realización de forma paralela, un buen punto de partida consiste en asignar cada thread a un píxel. Entonces, cada thread puede realizar la suma y multiplicación con total independencia del resto de threads, aunque se puede beneficiar de ellos a la hora de conseguir los datos que necesita, por ejemplo, mediante la utilización de memoria dinámica.

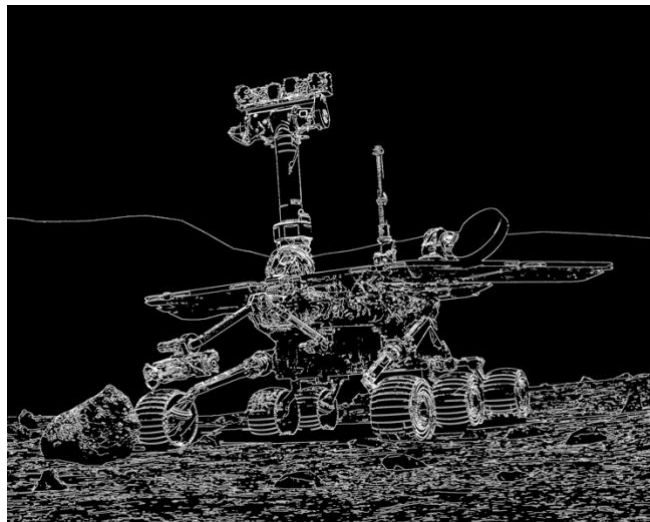
Objetivos parciales

- Ser capaz de realizar un código paralelo de *box filter*. Para su correcta evaluación, se testeará respecto a las soluciones de referencia presentadas con los filtros por defecto Laplace 5x5 y Gaussiano 9x9. Si cualquier píxel difiere en más de un cierto valor de umbral, se considerará un error. Esto implica tener que descubrir cómo gestionar los puntos denominados de borde (3 puntos). Como ayuda se puede comprobar las diferencias de imagen utilizando <https://rsmbl.github.io/Resemble.js/>
- Utilización de memoria de constantes (0,5 puntos).
- Utilización de memoria compartida y cooperación entre los diferentes threads (2 puntos).

- Correcta adecuación del número de bloques y de threads por bloque para diferentes tamaños de imagen incluyendo una mínima evaluación de prestaciones entre la que se puede mostrar escalabilidad, profiling, etc.... No es necesario tener en cuenta la arquitectura concreta de la tarjeta, pero si indicar las limitaciones sobre la arquitectura en concreto sobre la que se ha ejecutado (1 punto).
- Crear diferentes filtros para descubrir cómo conseguir nitidez, suavizado, detección de bordes y aristas (0.5 puntos). En los argumentos de entrada se puede pasar el identificador numérico de los filtros implementados para poder seleccionar el que se quiera. Como ayuda se puede consultar:

<https://desktop.arcgis.com/es/arcmap/latest/manage-data/raster-and-images/convolution-function.htm>

- El motivo de implementar varios filtros es para poder utilizarlos luego en un filtro complejo que puede estar compuesto de varios de ellos. En concreto se pide implementar un filtro complejo denominado “*Canny Edge Detector*” que usa algunos de los filtros ya desarrollados uno detrás de otro además de otras funcionalidades, entre las cuales debe aparecer una reducción, que se ejecutaran también como nuevos kernels (3 puntos).



Un *Canny Edge Detector* se compone de los siguientes pasos¹:

1. Pasar la imagen a escala de grises ya que el método está pensado ese formato.
2. Eliminar ruido con un filtro blur gaussiano (5x5).
3. Cálculo del gradiente en magnitud y dirección mediante filtros de sobel horizontal y vertical.
4. Supresión de los no-máximos en la dirección del gradiente (suprimir el valor de un pixel si tiene un vecino con un valor más grande en la dirección del gradiente).

¹ Una descripción más detallada del método puede encontrarse en Canny Edge Detection Step-by-step in Python: <https://medium.com/towards-data-science/canny-edge-detection-step-by-step-in-python-computer-vision-b49c3a2d8123>

5. Umbral doble de valores, identificando los píxeles como “fuertes”, “débiles” o irrelevantes para la imagen final para lo que habrá que calcular el máximo valor de los píxeles de la imagen.
6. Afinar los bordes mediante “*hysteresis thresholding*” (convertir píxeles “debiles” en “fuertes” si al menos uno de sus vecinos es “fuerte” para perfilar mejor los bordes).

Nota: las puntuaciones para cada objetivo parcial son las puntuaciones máximas que se pueden obtener si se cumplen esos objetivos. No se debe hacer un programa separado para cada objetivo, sino un único programa genérico que cumpla con todos los objetivos simultáneamente.

Nota: aquellos alumnos que cuenten con acceso al cluster de la URJC deberán mostrar además resultados de ejecución con tamaños de imágenes grandes. Se cuentan con imágenes de tamaño elevado (+200MB) en Aula Virtual de forma adicional que pueden servir de ejemplo. Igualmente, ya que se tendría acceso a múltiples tarjetas se deberá plantear una solución que ejecute varios kernels a la vez.

Entrega de prácticas

La entrega de prácticas se hará a través de Aula Virtual en las fechas anunciadas. Se debe entregar un único archivo *func.cu* con el código de toda la práctica, debidamente comentado y una memoria de la misma en formato pdf. La memoria debe incluir:

- Índice de contenidos y Autores.
- Descripción del código: incluyendo descripción de las principales funciones implementadas
- Análisis de las mejoras introducidas con evaluación de prestaciones, especificando tiempos relativos al uso de memoria compartida, memoria de constantes, tamaños de bloque, etc...
- Comentarios personales: incluyendo problemas encontrados, críticas constructivas, propuesta de mejoras y evaluación del tiempo dedicado.
- No incluir código fuente
- NO DESCUIDE LA CALIDAD DE LA MEMORIA DE SU PRÁCTICA. La memoria es tan imprescindible para aprobar la práctica, como el correcto funcionamiento de la misma.

Autoría de la práctica

La práctica se debe realizar en grupos de 2 personas, como máximo. El hecho de detectar copia en las prácticas expondrá a los alumnos a la posibilidad de una apertura de expediente disciplinario y expulsión. En caso de detectar copia, los alumnos afectados serán suspendidos en la TOTALIDAD de la asignatura. Una práctica será considerada copia en caso de que contenga la totalidad o una parte de la práctica de otro alumno.